TRASER-F: прогнозирование графов сцены

Тройки и локализация объектов на PSG4D-HOI

Репозиторий `lve-gh/SVG2`, ветка `forecasting`

18 октября 2026

Аннотация

TRASER (ECCV 2026) строит граф сцены по уже увиденному видео. Мы добавили к нему прогноз: по первым H секундам клипа и маскам объектов модель предсказывает отношения (субъект, предикат, объект) на следующие F секунд и положение каждого объекта, $H, F \in [1, 5]$ с. Документ описывает выбранную архитектуру TRASER-F и почему выбрана именно она, обзор современных подходов, протокол оценки, написанный код, что уже проверено и что остаётся сделать на сервере. Обучение на полных весах ещё не запускалось.

Содержание

1 Задача и данные **1**

2 Обзор современных подходов **2**

3 Архитектура TRASER-F **2**
3.1 Исходный TRASER 2
3.2 Схема 3
3.3 Прогнозатор латентов 3
3.4 Почему эта архитектура 3
3.5 Функция потерь и обучение 4

4 Протокол оценки **4**

5 Что написано **5**

6 Что проверено **6**

7 Запуск **7**

8 Открытые вопросы **7**

1 Задача и данные

Постановка.

Вход	кадры интервала $[T-H, T)$ с частотой 2 кадра/с, маски объектов на этих кадрах, классы объектов
Выход	`{"relationships": [[s, pred, o, [[t0, t1]]], ...]}` для секунд $H, \ldots, H+F-1$ клипа; для каждого объекта и каждой будущей секунды — бокс (c_x, c_y, w, h) и видимость
Объекты	видимые в наблюдаемой части; новые объекты в будущем не появляются (как в SGA)

Это соответствует режимам GAGS/PGAGS из scene graph anticipation (объекты даны, прогнозируются отношения) плюс локализация из scene graph *forecasting* (FDSG).

PSG4D-HOI. Часть PSG4D, построенная на HOI4D (без GTA-части):

- 2973 эгоцентричных видео по 20 с, 15 кадров/с (300 кадров), 1920 × 1080;
- 46 классов объектов и около 15 предикатов: *reachout, grasp, pickup, putdown, carry, carrywithbothhands, open, close, push, pull, press, cut, dump, binding*;
- субъект почти всегда `hand`; все исполнители (H1–H4) — люди, роборук нет.

В HOI4D действия размечены отрезками времени (`action/color.json`), маски объектов — цветные PNG (`2Dseg/shift_mask`). Из этого же PSG4D получал свои отношения («hand <действие> <объект>»), поэтому конвертер умеет собирать разметку и напрямую из HOI4D (раздел 5).

Особенность данных. Действия короткие: grasp ≈ 0,45 с, pickup ≈ 0,4 с. В одну секунду часто попадают 2–3 предиката подряд, и их порядок внутри секунды теряется. Это учтено в открытых вопросах (раздел 8).

2 Обзор современных подходов

Метод	Где	Как прогнозирует	Боксы	Что взяли в TRASER-F
STTran+/++, DSG-DETR+/++	базовые в SceneSayer	авторегрессионный трансформер	нет	авторегрессия накапливает ошибку
SceneSayer ODE/SDE	ECCV'24	динамика признаков пар объектов в латентном пространстве (NeuralODE/SDE)	да	**прогноз в латентах**, декодирование теми же головами
ImparTail	CVPR'25	SceneSayer/STTran + curriculum с маскированием неполной разметки	—	устойчивость к пропускам в разметке
FDSG	2025	NeuralSDE на запросах DETR, бокс = тождество + поправка	**да**	**бокс = последний** + Δ; метрики SGF с IoU ≥ 0,5
HyperGLM	CVPR'25	гиперграф + мультимодальная LLM	нет	LLM-декодер конкурентен в SGA
OOTSM	2025	LLM по текстовым графам: сначала объекты, потом отношения	нет	LLM даёт больше всего в mR (+21,9 mR@50)
FORESCENE	2025	латентная диффузия графов	нет	вероятностный прогноз — следующий шаг

Ни один из методов не работает с токенами траекторий объектов внутри видео-LLM. TRASER-F переносит идею SceneSayer/FDSG (латентная динамика, условие на горизонт, поправка к последнему состоянию) в собственное представление TRASER и оставляет LLM TRASER декодером, как в HyperGLM/OOTSM.

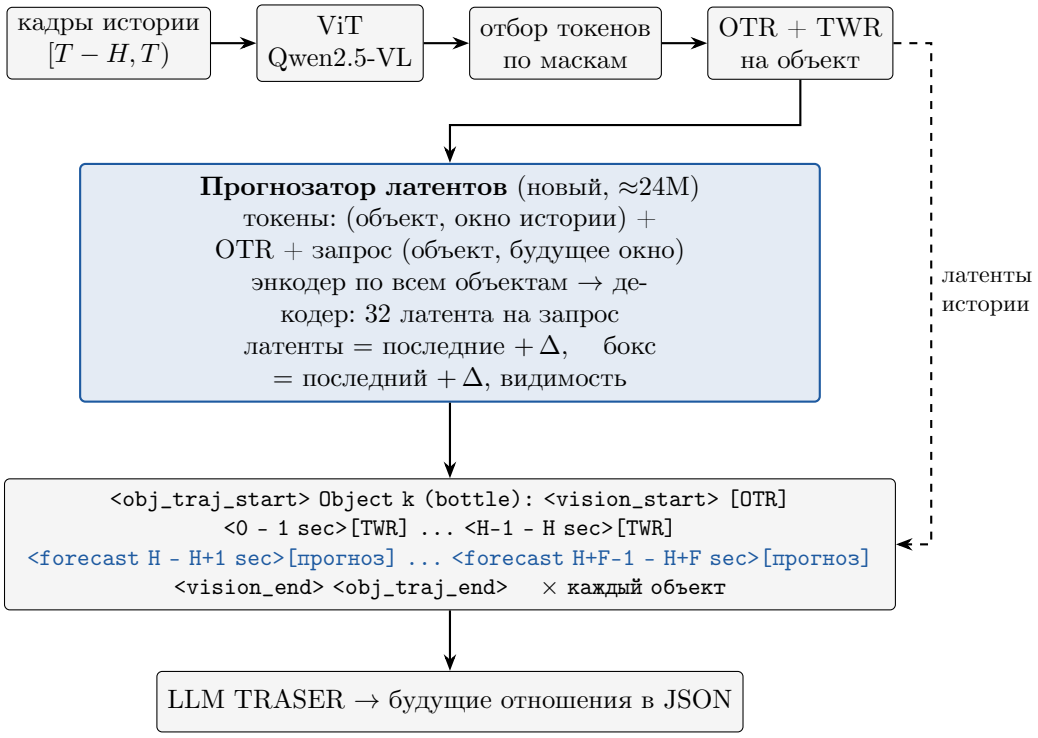
3 Архитектура TRASER-F

3.1 Исходный TRASER

Qwen2.5-VL-3B и два Perceiver-ресемплера. Токены ViT отбираются по покрытию маской объекта; для каждого объекта строится блок: OTR (вся траектория, 32 латента) и TWR (32 латента на каждое временное окно). LLM читает блоки объектов и выдаёт граф сцены в JSON.

Мы берём частоту 2 кадра/с: при временном патче 2 один грид ViT равен одной секунде, т. е. одному окну TWR.

3.2 Схема



3.3 Прогнозатор латентов

Класс `ObjectLatentForecaster` (`forecast/dynamics.py`).

- **Токены.** Для каждого объекта: по токену на окно истории (усреднённые латенты TWR после LayerNorm и проекции + бокс окна), токен OTR и по запросу на каждое будущее окно. К каждому токену добавляются эмбеддинги абсолютного времени, типа токена (история / запрос / OTR) и индекса объекта.
- **Энкодер.** Трансформер (4 слоя, $d = 512$, 8 голов, pre-norm) по *всем* токенам клипа: объекты видят друг друга.
- **Декодер.** 2 слоя: 32 обучаемых запроса, сдвинутых выходом энкодера для данного (объект, окно), смотрят на последние наблюдённые латенты объекта и дают $R = 32$ латента будущего окна.
- **Выходы.** $\hat{z}_t = z_{\text{last}} + W_{\text{out}}\, h_t, \quad \hat{b}_t = \text{clip}(b_{\text{last}} + \Delta b_t), \quad$ логит видимости. W_{out} и голова Δb инициализированы нулями: до обучения модель *в точности* равна базовому copy-last.
- Около 23,7M параметров при $d_{\text{LLM}} = 2048$.

3.4 Почему эта архитектура

1. **Прогноз в латентах TRASER.** Будущие слоты заполняются латентами того же вида, что LLM уже умеет читать, поэтому декодер графа TRASER переиспользуется как есть.
2. **Сразу весь горизонт, без авторегрессии.** Каждое будущее окно предсказывается напрямую по истории (запрос = объект + абсолютное время), ошибка на 5 с не накапливается.
3. **Взаимодействие объектов.** Рука, тянущаяся к бутылке, меняет будущее бутылки.

4. **Старт с copy-last.** Обучение отходит от инерции только там, где это помогает; модель не может стать хуже базового метода из-за плохой инициализации.
5. **Бесплатная плотная разметка.** На обучении будущие кадры известны: латенты, которые ресемплер строит по ним, служат целями дистилляции без дополнительной разметки.
6. **Нет утечки будущего.** ViT Qwen2.5-VL работает внутри одного временного грида (`cu_seqlens`), поэтому кодирование всего клипа даёт те же признаки истории, что кодирование одной истории (проверено тестом). При оценке модели подаются только наблюдаемые кадры.

3.5 Функция потерь и обучение

$$\mathcal{L} = \mathcal{L}_{\text{LM}}(\text{будущий JSON}) + \lambda_{\text{lat}} \left\| \text{LN}(\hat{z}) - \text{LN}(\text{sg}(z^{\text{teacher}})) \right\|_2^2 + \lambda_{\text{box}} \left(5\,\mathcal{L}_1 + 2\,\mathcal{L}_{\text{GIoU}} + \mathcal{L}_{\text{BCE}}^{\text{vis}} \right), \qquad \lambda_{\text{lat}} = \lambda_{\text{box}} = 1.$$

Цели z^{teacher} — латенты TWR реальных будущих окон, посчитанные без градиента.

Этап	Обучается	LR	Зачем
A, 20 эпох	только прогнозатор	10^{-4}	выучить динамику против замороженного TRASER: дёшево, учитель стабилен
B, 10 эпох	LLM	10^{-5}	адаптировать декодер к прогнозу
	merger	$2 \cdot 10^{-5}$	
	ресемплеры	$5 \cdot 10^{-5}$	
	прогнозатор	$5 \cdot 10^{-5}$	

DeepSpeed ZeRO-3 с offload, bf16, накопление градиента 4, косинусный LR. Клипы собираются на лету: случайные $H, F \in [1, 5]$; в 70 % случаев якорь выбирается так, чтобы в будущем окне начиналось отношение; клипы с пустым будущим сохраняются в 10 % случаев.

Режимы (`--forecast_mode`, один флаг): `latent` — основная модель; `copy` — в будущие слоты копируются последние латенты; `oracle` — настоящие будущие латенты (потолок); `prompt` — будущих слотов нет, LLM прогнозирует только по истории.

4 Протокол оценки

Данные. PSG4D-HOI, разбиение *по экземплярам объектов* (`C*/N*` в пути HOI4D), 75/10/15: тестовые объекты в обучении не встречаются.

Клипы. Каждую секунду каждого тестового видео, для $(H, F) \in \{1, 3, 5\}^2$. Основной результат — $H = 3$, $F = 3$. Все методы, включая базовые без обучения, оцениваются на одних и тех же клипах (общая функция `clip_targets`).

Метрики (`forecast/metrics.py`), по будущим секундам:

R, mR	полнота четвёрок (s, pred, o, секунда); mR — среднее по предикатам
P, F1	точность, чтобы не окупалось перечисление всех предикатов
R@k	полнота на k-й секунде вперёд (как быстро падает качество)
mIoU@k, ADE, FDE	прогноз бокса против реального
R_{loc}	как в SGF: тройка верна *и* боксы обоих объектов с IoU $\geq 0{,}5$

Базовые методы (`forecast/baselines.py`). Для троек: copy-last и марковская цепочка переходов предикатов; для боксов: copy и постоянная скорость. Им даётся истинный наблюдаемый граф, т. е. больше, чем видит модель, поэтому обойти их — содержательный рубеж.

ImparTail / SceneSayer. Их код привязан к Action Genome (боксы по кадрам, признаки Faster R-CNN). Два сравнения:

1. *На PSG4D-HOI:* обучить SceneSayerSDE (с curriculum ImparTail и без) на нашем разбиении в режиме PGAGS/GAGS, боксы из наших масок, 1 кадр/с; оценка `recall_at_k` на наших четвёрках.
2. *На Action Genome:* прогнать TRASER-F с прямоугольными масками из разметки AG по протоколу SGA ($F \in \{0{,}3; 0{,}5; 0{,}7; 0{,}9\}$, R@10/20/50 With/No constraint) и сравнить с опубликованными SceneSayerSDE, ImparTail, FDSG.

Гипотезы и абляции. Всё с одинаковыми данными, расписанием и оценкой; меняется один флаг.

ID	Гипотеза	Запуск
B0	инерция — сильный базовый метод	`baselines.py`, `--mode copy`
H1	LLM может прогнозировать по одной истории	`MODE=prompt`
H2	латентная динамика лучше инерции (основная модель)	`MODE=latent`
H2a	работает именно дистилляция латентов	`LAMBDA_LATENT=0`
H2b	боксы помогают отношениям	`LAMBDA_BOX=0`
UB	потолок при идеальной динамике	`evaluate.py --mode oracle`
H3	длина истории важна	одна модель, $H = 1 \dots 5$

Сравнение `oracle`/`latent`/`copy` разделяет ошибку на «динамику» и «декодирование»: если `oracle` ненамного лучше `copy`, узкое место в декодере.

5 Что написано

Весь код — новые файлы; исходный код TRASER не изменён.

Файл	Содержание
traser/traser_train/forecast/	
data_forecast.py	формат разметки, нарезка клипов, боксы из RLE-масок, ForecastDataset, коллатор
dynamics.py	прогнозатор латентов, лоссы дистилляции и боксов (L1, GIoU, BCE)
arrangement.py	сборка входа LLM: блоки объектов с будущими слотами, позиции, rope_deltas
step.py	отбор токенов по маскам для истории и будущего, метки времени
modeling_forecast.py	TRASERForecast(TRASER), загрузка из чекпойнта TRASER, инициализация под ZeRO-3
trainer_forecast.py	ForecastTrainer: группы LR, суммарный лосс, логирование составляющих
train_forecast.py	точка входа обучения, аргументы
metrics.py	разбор JSON, R/P/F1/mR/R@k, mIoU, ADE/FDE, R_{loc}
baselines.py	copy-last, марковская цепочка, боксы copy/velocity
evaluate.py	оценка: только кадры истории, генерация, метрики, jsonl
traser/data/prepare_psg4d_hoi.py	конвертер: psg4d (JSON PSG4D-HOI) и hoi4d (действия HOI4D)
traser/scripts/train_forecast.sh	этапы A/B через DeepSpeed
traser/tests/test_forecast_smoke.py	smoke-тест на крошечной случайной модели (CPU)
docs/FORECAST.md	описание на английском
docs/report/	этот отчёт (.tex и .pdf)

Формат разметки.

```
{"video": ".../image.mp4", "mask_json": ".../<seq>_rle.json", "fps": 15,
 "num_frames": 300, "objects": [{"id": 0, "name": "toy car"}, {"id": 1, "name": "right
    hand"}],
 "relationships": [[1, "grasp", 0, [[45, 52], [180, 190]]], ...]}
```

mask_json — список по кадрам, в каждом COCO-RLE для каждого объекта; интервалы отношений — в исходных кадрах.

6 Что проверено

Smoke-тест (python traser/tests/test_forecast_smoke.py; случайная Qwen2.5-VL с $d = 64$, синтетический клип 8 с: рука движется к бутылке) — проходит целиком:

1. датасет собирает клипы с нужным числом окон истории и будущего;
2. утечки будущего через ViT нет;
3. при инициализации прогноз в точности равен copy-last, в том числе (3b) при загрузке чекпойнта TRASER без прогнозатора;
4. градиенты доходят до прогнозатора; два шага ForecastTrainer.train();
5. оценка: генерация, разбор, метрики, базовые методы.

Найден и исправлен баг: стандартная инициализация HF не трогает in_proj_weight у nn.MultiheadAttention, и при загрузке из чекпойнта TRASER эти веса остались бы неинициализированными.

Реальное видео HOI4D (категория C1, игрушечная машинка; объекты toy car и right hand, 12 отрезков отношений, разметка через конвертер hoi4d): сборка клипа 3,7 с, грид $6 \times 40 \times 72$, длина собранной последовательности 341 токен, три шага обучения на крошечной модели прошли.

H/F	Метод	R, %	mR, %	P, %
3/3	copy-last	34,4	25,2	—
3/3	Markov	46,9	—	66,7
5/5	copy-last	20,2	—	—
5/5	Markov	34,5	—	—

Это не результат, а проверка, что конвейер работает: видео одно, а марковская цепочка обучена на том же видео. Для боксов постоянная скорость хуже copy в эгоцентричном видео (mIoU@1c 46 против 60): камера движется вместе с головой.

Не проверено: запуск с реальными весами 3B, DeepSpeed ZeRO-3 и flash-attn; конвертер `psg4d` на настоящих файлах PSG4D-HOI (формат восстановлен по скрипту визуализации авторов, файлы на SharePoint требуют входа).

7 Запуск

```
git clone -b forecasting https://github.com/lve-gh/SVG2 && cd SVG2

# 1. Данные: PSG4D-HOI или запасной вариант из действий HOI4D
python traser/data/prepare_psg4d_hoi.py psg4d --psg4d_json HOI/psg4d_hoi.json \
    --categories HOI/categories.json --hoi4d_root HOI/HOI4D_release \
    --mask_root HOI/HOI4D_mask --out_dir traser/data
python traser/data/prepare_psg4d_hoi.py hoi4d --hoi4d_root HOI4D_release \
    --annotation_root HOI4D_annotations --instructions HOI4D-Instructions \
    --out_dir traser/data

# 2. Проверка окружения и базовые методы
python traser/tests/test_forecast_smoke.py
cd traser && python -m traser_train.forecast.baselines \
    --train data/psg4d_hoi_train.json --test data/psg4d_hoi_test.json \
    --history_sec 3 --horizon_sec 3

# 3. Обучение (NUM_GPUS, MODE, LAMBDA_LATENT, LAMBDA_BOX, EXTRA_ARGS)
STAGE=A bash traser/scripts/train_forecast.sh
STAGE=B bash traser/scripts/train_forecast.sh

# 4. Оценка
python -m traser_train.forecast.evaluate --model checkpoints/traser_f_latent_stageB \
    --annotation data/psg4d_hoi_test.json --history_sec 3 --horizon_sec 3 \
    --mode latent --output preds.jsonl
```

8 Открытые вопросы

- **Файлы PSG4D-HOI.** Нужны `psg4d_hoi.json` и `categories.json` (SharePoint NTU, нужен вход). До этого — разметка из действий HOI4D.
- **3D.** Боксы сейчас 2D (`forecaster_box_dim=4`). Для 3D-версии TRASER достаточно задать размерность 3D-бокса и считать цели по глубине; остальной прогнозатор от этого не зависит. Нужен код командной 3D-адаптации.
- **Временное разрешение.** Если метрики по секундам упрутся в потолок — слоты по 0,5 с (`sample_fps 4`, полусекундные метки); прогнозатор не меняется.
- **Многовариантное будущее.** Прогнозатор детерминированный. Если H2 работает, но R@k быстро падает с k, — стохастический латент (SDE, как в SceneSayer/FDSG) и оценка best-of-N.

- **Сервер.** Нужны число и тип GPU (`nvidia-smi`), чтобы подобрать `NUM_GPUS`, батч и конфигурацию DeepSpeed.

Источники

- TRASER / SVG2: `https://github.com/uwGZQ/SVG2`
- PSG4D: `https://arxiv.org/abs/2405.10305`, `https://github.com/Jingkang50/PSG4D`
- SceneSayer (Towards Scene Graph Anticipation): `https://arxiv.org/abs/2403.04899`
- ImparTail: `https://github.com/rohithpeddi/ImparTail`
- FDSG: `https://arxiv.org/abs/2506.01487`
- OOTSM: `https://arxiv.org/abs/2509.05661`
- HyperGLM (CVPR 2025): `https://openaccess.thecvf.com/content/CVPR2025/papers/Nguyen_HyperGLM_HyperGraph_for_Video_Scene_Graph_Generation_and_Anticipation_CVPR_2025_paper.pdf`
- FORESCENE: `https://arxiv.org/abs/2503.06182`